```
import numpy as np

points = np.array(eval(input()))
centroids = np.array(eval(input()))

#function returns the manhattan distance between points p1
and p2
def calc_dist(p1, p2):
    """p1 and p2 are two tuples representing the points"""

    dist = 0

    #YOUR CODE GOES HERE

    return dist


#function returns the updated centroid for the points in lst
which represent a cluster
def update_centroid(lst):
    """lst is a 2d numpy array consisting of points in a cluster"""

    new_centroid = None

    #YOUR CODE GOES HERE

    return new_centroid

#function performs one iteration of k-means
def k_means(points, centroids):
    """points is a 2d numpy array consisting of points whereas
    centroids is a 2d numpy array consisting of initial
centroids"""


    # clusters[i] stores all the points with the cluster label i
    clusters = [[] for i in range(len(centroids))]

    for pnt in points:
        #tmp[i] stores the distance of pnt from centroid[i]
        tmp = _____
        for i,centroid in enumerate(centroids):
```

```
        tmp[i] = _______  #calculate the distance between
point and centroid

      cluster_id = _____(tmp)  #index of the centroid from
which pnt is closest
      clusters[cluster_id].append(pnt)


   #update the centroid of each cluster
   new_centroid = []
   for cluster in clusters:
      new_centroid.append(_______) #update the centroid of
cluster using update_centroid function we defined earlier

   return np.array(new_centroid)

for i in range(5):
   centroids = ______  #perform ith iteration on data to
update clusters using k_means function we defined earlier
   print(np.round(centroids,2))
```

Here is the complete and updated code with all the missing logic filled in:

Python

```python
import numpy as np

points = np.array(eval(input()))
centroids = np.array(eval(input()))


# function returns the manhattan distance between points p1 and p2
def calc_dist(p1, p2):
  """p1 and p2 are two tuples representing the points"""

  # Manhattan distance is the sum of absolute differences
  dist = np.sum(np.abs(np.array(p1) - np.array(p2)))

  return dist


# function returns the updated centroid for the points in lst which represent a cl
def update_centroid(lst):
  """lst is a 2d numpy array consisting of points in a cluster"""
```

```
    # Calculate the mean across the rows (axis 0) to find the center
    new_centroid = np.mean(lst, axis=0)

    return new_centroid


# function performs one iteration of k-means
def k_means(points, centroids):
    """points is a 2d numpy array consisting of points whereas

    centroids is a 2d numpy array consisting of initial centroids
    """

    # clusters[i] stores all the points with the cluster label i
    clusters = [[] for i in range(len(centroids))]

    for pnt in points:
        # tmp[i] stores the distance of pnt from centroid[i]
        tmp = [0] * len(centroids)
        for i, centroid in enumerate(centroids):
            tmp[i] = calc_dist(pnt, centroid)  # distance between point and centroid

        cluster_id = np.argmin(
            tmp
        )  # index of the centroid from which pnt is closest
        clusters[cluster_id].append(pnt)

    # update the centroid of each cluster
    new_centroid = []
    for cluster in clusters:
        new_centroid.append(
            update_centroid(cluster)
        )  # update using update_centroid

    return np.array(new_centroid)


for i in range(5):
    centroids = k_means(points, centroids)  # perform ith iteration of k_means
    print(np.round(centroids, 2))
```

Breakdown of the missing code:

- `calc_dist`: `np.sum(np.abs(np.array(p1) - np.array(p2)))` computes the element-wise absolute differences and sums them to calculate Manhattan distance.
- `update_centroid`: `np.mean(lst, axis=0)` computes the mean along columns to locate the cluster's center point.

- `tmp` initialization: `[0] * len(centroids)` creates a placeholder list matching the number of centroids.
- **Distance assignment:** `calc_dist(pnt, centroid)` calls the Manhattan distance calculation.
- `cluster_id` : `np.argmin(tmp)` finds the index corresponding to the minimum distance.
- **Centroid list update:** `update_centroid(cluster)` recalculates the centroid for each newly formed cluster.
- **Main loop execution:** `k_means(points, centroids)` feeds current centroids into the function to iteratively refine them.

A higher Dunn Index value indicates:

Select any ONE
Better cluster separation and compactness.

Higher noise levels in the data.

Poor clustering quality.

A need for more clusters.

Better cluster separation and compactness.

Why this is correct:

The **Dunn Index** measures clustering quality using the ratio of minimum inter-cluster distance to maximum intra-cluster distance:

$$\text{Dunn Index} = \frac{\text{Minimum distance between two different clusters}}{\text{Maximum distance between points in the same cluster}}$$

- **Higher inter-cluster distance** $\rightarrow$ Clusters are well-separated from each other.
- **Lower intra-cluster distance** $\rightarrow$ Points inside each cluster are tightly packed (compact).

Therefore, a higher Dunn Index represents better overall clustering performance.

3. Elbow point
Save for later
In the Elbow Method graph, what does the "elbow point" represent?

Select any ONE
The point where the algorithm converges.

The point where the sum of squared distances becomes zero.

The point where the silhouette score reaches its maximum.

The optimal number of clusters where the rate of decrease in sum of squared distances slows down.

The optimal number of clusters where the rate of decrease in sum of squared distances slows down.

Why this is correct:

The **Elbow Method** plots the Within-Cluster Sum of Squares (WCSS or sum of squared distances) against the number of clusters k.

- **Before the elbow point:** Adding more clusters significantly reduces WCSS because large, loose clusters are split into smaller ones.
- **At the elbow point:** The curve bends sharply, marking the optimal tradeoff between model simplicity (k) and cluster variance.
- **After the elbow point:** Adding additional clusters yields diminishing returns, only reducing WCSS at a much slower rate.

4. Merging strategy in agglomerative
Save for later
In agglomerative hierarchical clustering, which merging strategy starts by treating each data point as an individual cluster and then repeatedly merging the closest clusters until the desired number of clusters is achieved?

Select any ONE
Complete linkage

Average linkage

Single linkage

Ward linkage

Single linkage FasterCapital

Why this is correct:

While all agglomerative methods follow a bottom-up approach (treating each point as its own cluster initially), they differ in how they define the distance between two clusters:

- **Single linkage:** Defines the distance between two clusters as the distance between their **two closest points** (also known as the *nearest-neighbor* method). FasterCapital

Comparison with other linkage strategies:

- **Complete linkage:** Measures distance between the two *farthest* points in the clusters. FasterCapital
- **Average linkage:** Measures the *average* distance between all pairs of points across the clusters. FasterCapital
- **Ward linkage:** Measures distance based on the increase in total *within-cluster variance* when clusters are merged. FasterCapital

5. Assumption of Elliptic Envelope
Save for later
The Elliptic Envelope method assumes that the inlying data follows:

Select any ONE
Linear distribution.

Normal distribution.

Uniform distribution.

Exponential distribution.

Normal distribution.

Why this is correct:

The **Elliptic Envelope** method (often implemented via `EllipticEnvelope` in scikit-learn) assumes that the normal/inlying data follows a **multivariate Gaussian (Normal) distribution**.

It works by fitting a robust covariance estimate to the central data points, creating an elliptical boundary (hyperellipsoid) around them. Any points that fall far outside this ellipse are flagged as anomalies or outliers.

6. Value of LOF
Save for later
In the context of the Local Outlier Factor (LOF) method, a data point with an LOF value greater than 1 indicates that the point:

Select any ONE
Is an inlier within a dense cluster.

Is an outlier with low local density.

Deviates significantly from the mean.

Has few neighbors within the epsilon radius.

Is an outlier with low local density.

Why this is correct:

The **Local Outlier Factor (LOF)** measures the local density deviation of a given data point relative to its k-nearest neighbors:

- $\text{LOF} \approx 1$: The point has a density similar to its neighbors (it is an **inlier**).

- $\mathrm{LOF} < 1$: The point has a higher local density than its neighbors (it is deeply embedded inside a dense cluster).
- $\mathrm{LOF} > 1$: The point has a lower local density than its neighbors, meaning it is significantly more isolated than the points around it (it is an **outlier**).

7. t-SNE in high dim
Save for later
Why is t-SNE particularly effective in visualizing clusters of high-dimensional data?

Select any ONE
It creates a linear mapping of the data.

It amplifies the variance of the data.

It focuses on the global structure of the data.

It preserves local pairwise relationships.

It preserves local pairwise relationships.

Why this is correct:

t-SNE (t-Distributed Stochastic Neighbor Embedding) converts high-dimensional distances between data points into conditional probabilities that represent similarities. It specifically optimizes low-dimensional representations to keep points that are close together in high-dimensional space close together in the visualization.

- **Local pairwise similarities:** t-SNE focuses heavily on preserving local neighborhood structures, making it exceptional at revealing distinct clusters.
- **Non-linear mapping:** Unlike PCA (which creates a linear mapping and focuses on maximizing variance/global structure), t-SNE uses a non-linear approach that resolves complex geometric patterns and manifolds.

8. Ideal Clustering
Save for later

For an ideal cluster, which of the following statements is true?

Inter cluster distance should be high

Inter cluster distance should be low

Intra cluster distance should be high

Intra cluster distance should be low

Both of the following statements are **TRUE** for an ideal cluster: Medium

- **Inter cluster distance should be high** (Clusters should be far apart / well-separated from each other). Medium
- **Intra cluster distance should be low** (Points within the same cluster should be tightly packed / compact). Medium

Detailed Breakdown:

1. **Inter-Cluster Distance (High)**:
   - Measures the separation **between** different clusters. Medium
   - A **high** value ensures distinct grouping with minimal overlap between clusters.
2. **Intra-Cluster Distance (Low)**:
   - Measures cohesion/compactness **within** a single cluster. Medium
   - A **low** value ensures that all data points assigned to the same cluster are as similar as possible.

9. K-means initialisation
Save for later
Which of the following can arise from poor initialization in K-Means clustering?

Select any ONE
Slower convergence to the optimal solution

Sensitivity to the number of clusters (K)

Increased risk of convergence to a local minimum

Improved robustness against outliers

Increased risk of convergence to a local minimum

Why this is correct:

K-Means uses a greedy iterative approach to minimize the Within-Cluster Sum of Squares (WCSS). Because the objective function is non-convex, the final output depends heavily on where the initial centroids are placed:

- **Bad Initialization:** If initial centroids are placed poorly (e.g., two initial centroids placed within the same natural cluster), the algorithm can easily get stuck in a **suboptimal local minimum**, leading to poor cluster quality.
- **Solution:** Algorithms like **K-Means++** were developed specifically to choose initial centroids that are far apart, reducing the risk of getting stuck in local minima.

Why other options are incorrect:

- **Sensitivity to K:** Sensitivity to K is an inherent property of the K-Means algorithm itself, regardless of how centroids are initialized.
- **Improved robustness against outliers:** Poor initialization makes the performance worse, not better or more robust against outliers (K-Means remains inherently sensitive to outliers).

10. DBSCAN Noisy Pts
Save for later

How does DBSCAN handle noisy data points?

Select any ONE
By assigning noisy points to the nearest cluster

By identifying core points, border points, and treating isolated points as noise

By performing outlier removal before clustering

By creating a separate cluster for noisy points

By identifying core points, border points, and treating isolated points as noise

Why this is correct:

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) categorizes every data point based on two parameters— `eps` (radius) and `minPts` (minimum density):

- **Core points:** Points with at least `minPts` within their `eps` -neighborhood.
- **Border points:** Points within the `eps` -neighborhood of a core point, but with fewer than `minPts` in their own neighborhood.
- **Noise points:** Points that are neither core points nor border points.

Instead of forcing isolated points into a cluster or creating a separate cluster for them, DBSCAN explicitly marks them as **noise/outliers** (typically assigned a cluster label of `-1`).

11. PCA Steps
Save for later
Which of the following are steps involved in performing PCA?
Scaling the features to a similar range.
Compute the cosine similarity between each pair of data points.
Calculating the covariance matrix of the data.
Choosing the optimal number of clusters.

The steps involved in performing PCA are:

- Scaling the features to a similar range.
- **Calculating the covariance matrix of the data.**

Detailed Breakdown:

1. **Scaling the features to a similar range (Correct):**
    - PCA is highly sensitive to the scale of features because it relies on variance. Standardizing features (e.g., zero mean, unit variance) ensures that variables with larger numeric scales do not unnaturally dominate the principal components